

CS 214 Spring 2026

Project IV: A simple chat server

David Menendez

Due: Friday, May 1, at 11:59 PM (ET)

For this project, you and your partner will design and implement a simple Internet chat server. Your program, `chatd`, will operate a chat room for one or more client programs, forwarding messages from each participant to the others, and answering simple queries.

This project will provide experience of

- Using the sockets interface to accept connection requests
- Reading and writing messages in a simple application-level protocol
- Using multithreading or poll to react to multiple asynchronous events
- If multithreading, the use of mutual exclusion to safely access shared data

1 Overview

Your program `chatd` accepts one argument: the port on which it will listen for incoming connection requests. Once it has established its listening socket, it will operate indefinitely.

Clients will connect to the server, choose a unique screen name, and then send messages. A chat message sent by a client will be forwarded to all clients connected to the server. Clients may also request the list of screen names for the currently connected users or send messages to a specific user.

2 Protocol

`chatd` waits for clients to connect. Once a client has connected, it will send a NAME message to establish a screen name. If the name is not in use, `chatd` will reply with a welcome message, or with an error.

2.1 Concepts

Each user connected to `chatd` must choose a *screen name* before sending messages. The screen name can be 1–32 characters and consists of letters, digits, the hyphen, and the underscore. Users indicate their screen name when logging in, and two users may not have the same screen name.

Users may also set a *status*. User statuses are reported by the server in response to WHO queries. The status is 0–64 characters, and may use any characters in the range 32–126.

A *room* is a named collection of users. In this project, we have one room, named **#all**.

A *message* is a string of 1–80 characters in the range 32–126. Messages have a sender and recipient, either of which may be a user or room.

Note that clients can use the sender and recipient fields to distinguish (1) messages that are responses from the server (sender **#all**), (2) general messages sent by users (recipient **#all**), and (3) private messages from another user.

2.2 Message types

Our protocol uses five types of messages, each of which has additional fields. The message types are:

NAM Sent by a client to choose a screen name. One field: the screen name. The server will respond with a welcome message or **ERR**.

SET Sent by a client to establish a status. One field: the status.

MSG Sent by the client or the server. Three fields: the sender (ignored for messages sent by the client), the recipient, and the message body (note: messages from the server may exceed 80 characters).

WHO Sent by the client to request information about a user or all members of a room. One field: a screen name or room name.

ERR Sent by the server to reject a message or request. Two fields: an error code and an error explanation.

2.3 Message format

Messages are divided into several fields, terminated by vertical bars (|).

All messages begin with three fields:

1. The protocol version (always 1)
2. The message code (always three characters)
3. The message body length (in bytes; up to five digits)

The message body length is the total number of bytes in the message following the bar after the length field, and including the final bar after the last field.

It is an error for a message to not end with a vertical bar. The last field may include vertical bars as part of a message or status; these are not treated specially by the protocol. To avoid deadlock, clients and servers should not read more than the number of bytes specified in the length field before taking action.

2.4 Message flows

After connecting, a client must send **NAM** to choose a screen name. If the name is not in use, the server will add the user to **#all** and send a message from **#all** to the user indicating success or giving welcome.

```
>> 1|NAM|4|Bob|
<< 1|MSG|30|#all|Bob|Welcome to the chat!|
```

Once the user has been added, they will be visible to others and can receive messages sent to them or to the room.

Setting status A user can send SET to change their status. The server will update that user's status line. If the status is non-empty, the server will send a message from #all to #all indicating the status change.

```
>> 1|SET|17|Smiling politely|
<< 1|MSG|40|#all|#all|Bob is now "Smiling politely"|
```

Sending messages A user can send messages to #all, or to specific users. The server will forward the message to all users or the specific user, as appropriate, making sure to change the sender field to match the client. (That is, users are not permitted to masquerade as each other.)

For example, a message from Bob to Alice:

```
>> 1|MSG|28||Alice|Private message to Alice|
<< 1|MSG|35|Bob|Alice|Private message to Alice|
```

The server message is sent only to Alice.

A message from Bob to everyone:

```
>> 1|MSG|20||#all|Hello, world!|
<< 1|MSG|23|Bob|#all|Hello, world!|
```

The server message is sent to all connected users who have joined #all.

Querying users A user can send WHO to request information about a specific user, or about all users in the room. The server will respond with a message to the user from #all providing the screen name and status information (if any).

```
>> 1|WHO|6|Alice|
<< 1|MSG|33|#all|Bob|Alice: I was here first|
>> 1|WHO|6|Carol|
<< 1|MSG|20|#all|Bob|No status|
```

If the user has set a status, the response will be the screen name and status, separated by a colon. If the user has no status, the response will be "No status".

When asking about all users in the room, the response will be the screen name and status for each user, separated by newlines. For users with no status, only the screen name is printed.

```
>> 1|WHO|5|#all|
<< 1|MSG|61|#all|Bob|Alice: I was here first\nBob: Smiling politely\nCarol|
```

User information is printed in no particular order.

Note that, while user messages are limited to 80 characters, the server may send messages that are much longer, up to the limits of the 5-digit length field.

Table 1: Defined error codes

Code	Explanation
0	Unreadable
1	Name in use
2	Unknown recipient
3	Illegal character
4	Too long

2.5 Errors

Table 1 lists the error codes defined by the protocol.

The server sends error 0 if the protocol message is ill-formed or of an unknown protocol version or message type.

The server sends error 1 if the user requests a screen name that is in use.

The server sends error 2 if the user sends a message to or requests information about a screen name not currently in use.

The server sends error 3 if the user requests a screen name, sets a status, or sends a message using a character not allowed.

The server sends error 4 if the user requests a screen name, sets a status, or sends a message that is too long.

Error 0 is fatal, and the server should close the connection after sending it. The other errors are recoverable, and the server should wait for corrections.

3 Submission

Submit a Tar archive containing a directory, P4, in which are:

- Your source code file(s), including testing code
- Your make file
- Your AUTHOR file
- Your README

The AUTHOR should be a plain text document containing ONLY the NetIDs of the two partners, separated by whitespace, with no additional text or information.

Your README should be a plain text document containing the names and NetIDs of both partners.

Your test plan should be part of the README, and detail your testing strategy and test cases. Describe the scenarios you considered it important to check and how you performed those checks. Note that having a good test suite is an excellent way to detect errors that have been introduced or reintroduced into your code.